Copperbelt University
Computer Science Department

Introduction to Operating Systems

By Dr Derrick Ntalasha

CS 225: Introduction to Operating Systems

ASSIGNMENT 1

DUE DATE END OF TERM 2

Design and Implementation of a Mini Operating System for a Smart Emergency Response Center

A city has established a Smart Emergency Response Center (SERC) that coordinates ambulances, fire services, and police units in real time. The center runs on a lightweight embedded system with limited CPU and memory resources.

Due to frequent system overloads during emergencies, the city council has commissioned you to design a Mini Operating System (Mini-OS) module in C that efficiently manages:

- Multiple emergency tasks arriving concurrently
- Limited CPU processing time
- Restricted main memory
- Critical resource sharing without system deadlocks

Your Mini-OS will not replace a full operating system, but will simulate and implement core OS mechanisms to demonstrate how an operating system manages competing tasks under pressure.

Task

You are required to design, implement, and demonstrate a C-based software application that simulates a Mini Operating System for the Smart Emergency Response Center.

The system must manage emergency tasks (e.g., ambulance dispatch, fire alert processing, police coordination) as processes and apply operating system principles to ensure efficiency, fairness, and reliability.

System Requirements:

Your Mini-OS application must implement at least FOUR (4) of the following Operating System components:

1. Process Management

- Create, execute, suspend, and terminate processes using C
- Represent process states (New, Ready, Running, Waiting, Terminated)
- Maintain a Process Control Block (PCB) structure

2. CPU Scheduling

- Implement and compare at least TWO (2) scheduling algorithms:
    - First Come First Serve (FCFS)
    - Shortest Job First (SJF)
    - Priority Scheduling (Emergency level–based)
    - Round Robin
- Display scheduling metrics:
    - Waiting time
    - Turnaround time

    - CPU utilization

3. Memory Management
    - Simulate memory allocation using:
        - First Fit / Best Fit / Worst Fit
        - Paging (optional bonus)
    - Track memory usage and fragmentation
4. Inter-Process Communication (IPC)
    - Implement IPC using:
        - Pipes
        - Message queues
        - Shared memory
    - Demonstrate coordination between emergency processes
5. Deadlock Handling
    - Simulate resource allocation (e.g., communication channels, vehicles)
    - Implement deadlock prevention, detection, or avoidance (Banker's Algorithm)
6. File Management
    - Log emergency events and system decisions to files
    - Implement basic file read/write operations in C

Application Constraints:

- Programming Language: C (mandatory)
- Environment: Linux or UNIX-based system recommended
- Libraries: Standard C libraries and POSIX system calls
- Interface: Console-based menu system (clear and interactive)

Functional Expectations

The application should allow a user to:

- Create emergency tasks with different priorities and resource needs
- Select a CPU scheduling algorithm
- Observe how tasks are scheduled and executed
- View system status (process table, memory usage, resource allocation)
- Detect and handle deadlocks
- View system logs stored in files

Deliverables

1. C Source Code (well-commented and modular)
2. Executable Program
3. Technical Report (8–12 pages) including:
    - Problem analysis and scenario mapping
    - System architecture and data structures
    - Algorithms implemented
    - Screenshots of execution
    - Challenges, limitations, and improvements
4. Live Demonstration / Presentation